Analysis of Football Data using Python

PREPARED BY
TALHA IBNA BANNA | ROLL: 13

Final project on

Analysis of Football Data Using Python

Course name: Business Analytics

Course Code: K301

Submitted to

Aumit Ahsan

Lecturer

Institute of Business Administration

University of Dhaka

Submitted by

Talha Ibna Banna

Roll: 13

BBA 30th

Section A

Date of Submission

23/08/2025

Table of Contents

2. Background 1
2.1. Dataset Metadata: 1
3. Descriptive Statistics and Exploratory Data Analysis 1
3.1. Data Cleaning and Manipulation for Exploratory Data Analysis and Descriptive Statistics 1
3.2. Difference in Average Goals across Divisions 2
3.3. Average Elo Per Year of Major Divisions Across 25 Years 2
3.4. Correlation between Home Team Wins and Significant Numerical Columns 2
3.5. Distribution of Goals Across Major Leagues and Outliers 3
3.6. Trends Related to Manchester United 3
4. Additional Insights from Pandas 4
4.1. Do home teams have an advantage? 4
4.2. Has This Home Team Advantage Trend Changed Recently? 4
4.3. Teams named United in EPL 4
4.4. Highest scoring games of Manchester United 5
5. Modeling 5
5.1. Modeling Part 1 (Preserving Maximum Number of Rows) 5
5.2. Modeling Part 2 (Preserving Maximum Number of Features) 7
5.3. Model Summary 8
6. Reflection 8
References 8
Appendix 9

Table of Figures

Figure 1: Average Goals by Division 2
Figure 2: Average Elo across major divisions 2
Figure 3: Heatmap of correlation 3
Figure 4: Displot of total goals per game 3
Figure 5: Boxplot of goals across divisions 3
Figure 6: Line charts of Man United and Man City elo 3
Figure 7: Relplots of goal trends 4

Table of Tables

Table 1: Highest Scoring Man United Games 5
Table 2: Summary of Model Performance Metrics 1 6
Table 3: Summary of Performance Metrics 2 7
Table 4: Comparison of Model Accuracy 8

1. Background

While selecting the dataset for this project, I first looked into what I like and then jumped into the practicality of the data. I decided to go with football, a sport I have been following since I was a child. After looking at many datasets that do not align with the criteria, I found a dataset which I can explore combining my love for the sport and my affinity towards data analysis. Apart from personal reasons, research on football is relevant given the sport's rising popularity with top European clubs alone contributing €38 billion in revenue during the 2023/24 season (NY Times, 2025). The research can appeal to those who want to validate their thoughts with data and ones looking for predictive models to predict game outcomes.

The **dataset** titled *Club Football Match Data* contains match data from 42 different leagues across 27 countries, including major football leagues like the English Premier League, La Liga, Bundesliga and so on. The timeline of the data spans from the 2000/01 season to the most recent results of the 2024/25 season. The dataset, compiled by Gábor (2025), uses data from Football-Data.co.uk and ClubElo.

1.1. Dataset Metadata:

- **Source:** GitHub repository by Adam Gábor (https://github.com/xgabora/Club-Football-Match-Data-2000-2025)
- **Dimensions:** *230557 rows x 48 columns*
- **Key Features:** Division, MatchDate, MatchTime, HomeTeam, AwayTeam, HomeElo, AwayElo, (Elo shows relative strength of teams), Form3Home, Form3Away (Shows points gathered by home and away team over the last 3 games), FTHome, FTAway, FTResult, in-match stats (shots, fouls, corners, cards), OddHome, OddAway (Betting odds of victory)[1]
- **Time Span:** 2000 to 2025
- **Formats**: Primarily numerical (Elo, stats, odds) and categorical (results, team names)
- **Target Variable**: FTResult (Shows whether the home or away team won at full time)

2. Descriptive Statistics and Exploratory Data Analysis

My analysis started by simply looking at the dataset. After loading the dataset from GitHub raw file link into df, I checked the head of the data and immediately saw missing values across multiple features. As the dataset has diverse information across 42 different leagues, the analysis required extensive cleaning. I will present my cleaning operations for each part of the analysis as they come.

2.1. Data Cleaning and Manipulation for Exploratory Data Analysis and Descriptive Statistics

- Changed data type of MatchDate column to datetime format using pd.to_datetime
- Checked the percentage of null values in each column utilizing isnull
    - Dropped 13 features with either highly technical information like HandiAway (Away Team Win Odd with the given handicap size) and C_PHB. These features had almost or over 50% null values as smaller leagues and records from the past lack these data.
- Dropped null values if *null rows* < 2% of the column using dropna and comparison operators
- Made a subset named df_exploratory with 11 major leagues for significant EDA results. Here the codes 0/1 in division column refer to the 1st or main football division within that country.
- Dropped 12 more columns not relevant for EDA and was left with *83637 rows* and *27 features*
- Added additional columns like **Total Goals** and **Elo Difference** to see how many goals were scored in a match and difference in elo between teams.

Data should tell a story. For the exploratory analysis part that comes ahead, I have analyzed how different football leagues compare today and how they have evolved over the years, looked for iconic matches and delved into trends surrounding my favorite football club, Manchester United, and have compared their performance over the years with their rivals, Manchester City.

[1] Detailed column descriptions at GitHub and in appendix

2.2. Difference in Average Goals across Divisions

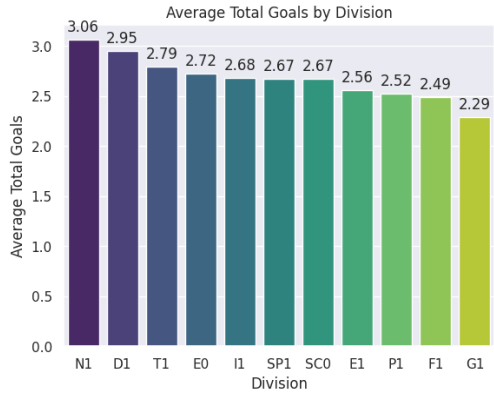


Figure 1: Average Goals by Division

We first found the **average goals per game** for the 11 divisions in our exploratory dataset. From the barplot created in seaborn it is evident that there is a significant difference in average goals scored in a match across different leagues. N1 **(Eredivisie)**, the primary league in Netherlands appears to be the most competitive in terms of average goals seen in a match while G1 **(Super League Greece),** the primary league in Greece, seems to be the least competitive in terms of goals. E0, The English Premier League sits at a relatively high 2.72 goals per match followed by I1 (Italian Seria A) and the SP1 (Spanish La Liga)

2.3. Average Elo Per Year of Major Divisions Across 25 Years

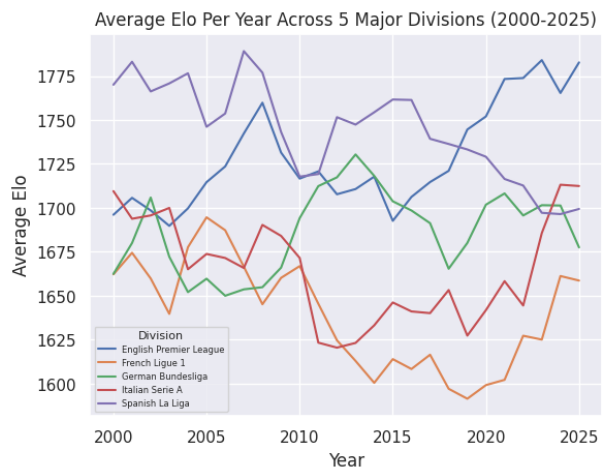


Figure 2: Average Elo across major divisions

The Elo feature in our dataset indicates a club's relative strength. Here we measure the average Elo of football's top 5 leagues as per Union of European Football Associations (League names in the legend).

To prepare the data for this section we first had to create an *AvgElo* column by finding the average of the 2 teams' elo in a given match, make a Year column by extracting the Year value from *MatchDate*, subset a df with the 5 major divisions, change division codes for full names for clarity, find average elo for each division and lastly we create a lineplot and separate the *Divisions* by *hue*.

We have some fascinating observations here. It seems that back in the early 2000s, **La Liga** dominated the football scene with the presence of strongest teams (By Elo) in league. This seems to align with the fact that global superstars like Zidane, Ronaldinho and Ronaldo (R9) all played in La Liga in that period. The rest of the leagues started slowing down but the **English Premier league** saw consistent rise in strength over the last 2 decades, followed by Bundesliga**. League 1** is consistently at the bottom of the top 5 leagues which is corroborated by the league's reputation of being uncompetitive. The rise of the **English Premier League** has been significant, and it is the strongest league in the world by Elo currently. This is in line with its popularity as even Bangladesh has developed a massive EPL fanbase over the recent years. This is further corroborated by the competitiveness in EPL. La Liga has fallen in recent years as only 3/4 teams dominate the league table. While in the EPL the top 7/8 teams are all competing for the major trophies, evident in the Europa League final last year where Manchester United and Tottenham played even though they were both in the bottom half of the league. The results here align with UEFA and Opta Power rankings, both of which currently lists EPL as the strongest football league and ranks French Ligue 1 at 5th (Furniss, 2025).

2.4. Correlation between Home Team Wins and Significant Numerical Columns

To prepare the data here we replace home victories with 1, and draws and losses with 0 in the FTResult column, and put the new values into a new HomeWin column with binary values (1 or 0). **A heatmap** of numerical features reveals a positive relationship between the **Elo difference** and **home wins**, with a correlation coefficient of 0.33, indicating that a larger Elo advantage for the home team increases the likelihood of a home victory, providing a potential feature for predictive models. **HTHome** (Home team goals at half time) has a decent positive correlation with winning and **HTAway** (Away team goals at half

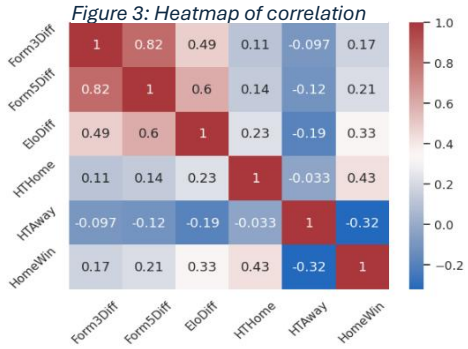


Figure 3: Heatmap of correlation

time) has a negative correlation, indicating that home teams that score half time have a higher chance of getting a win. Lastly, the **Form3/5Diff** features aggregate difference in number of points gathered by each team over the last 3/5 games and we see that correlation between win and form over 5 days being higher, indicating that teams with better form over longer periods may be more poised to win.

2.5. Distribution of Goals Across Major Leagues and Outliers

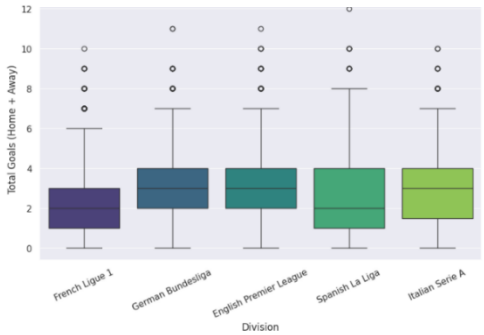


Figure 5: Boxplot of goals across divisions

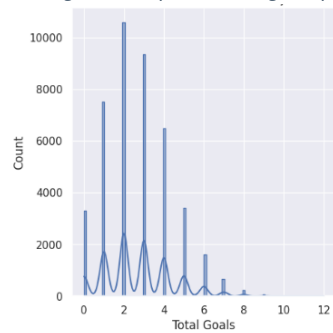


Figure 4: Displot of total goals per game

We have previously seen the average goals scored per game across the top 5 leagues. Here, we create a **boxplot** of total goals across divisions. It reveals that French league is at the bottom in terms of average goals in a match. This **corresponds to their low Elo over the years** as well. La Liga has the highest diversity in terms total goals in a match with a high IQR of around 3. But, the boxplot is positively skewed, indicating most games end in a relatively low score. Another **displot** of Total Goals across the 5 major leagues further reveals that most games have historically ended with 2 goals among the teams. The outliers of total goals refer to iconic games like the highest scoring EPL game where Portsmouth and Reading shared 11 goals back in 2007 and Real Madrid's 10-2 victory over Rayo Vallecano in 2015.

2.6. Trends Related to Manchester United

When picking this dataset, this was one of the parts I was looking forward to the most as a lifelong Manchester United fan. We had to run some **significant cleaning operations** as the dataset is diverse and I want to particularly bring out the data related to Manchester United, it's rival Man City and the EPL.

2.6.1 The Shifting Manchester Rivalry

Figure 6: Line charts of Man United and Man City elo

To create these line charts, we first had to separate Man United and Man City's Elo from the Home and Away Elo columns and make 2 data subsets for the 2 clubs first. Then we grouped mean of this Elo by year. The Elo levels tell the story of how Man United has progressed. Strength of Man United peaked at 2008, around the time when the club won the **Champions League in 2008**, marking their dominance in this era.

The **steep decline** in Elo around **2013** is when Sir Alex Ferguson left his managerial role at Man United after 26 years. The sudden change in manager was a big shock for the team and we have not yet quite recovered according to the graph. The absolute **low in 2025** is because Manchester United has had their **lowest league position (16th)** in the history of the league in 2025. The story is very different when it comes to **Man City**, lifelong rivals of Manchester United. They have seen a meteoric rise in terms of strength between 2020 and 2024 before seeing a dip in 2025. This can be attributed to the fact that Manchester City won **4 consecutive Premier League titles** between 2020 and 2024 but finally **lost to Liverpool in 2025**.

2.6.2. Manchester United and The Premier League

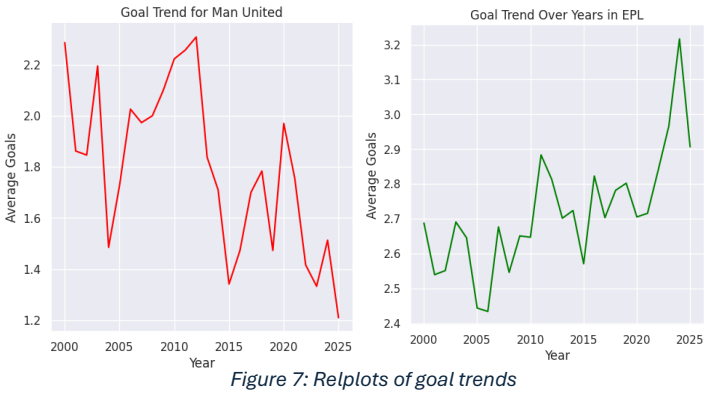


Figure 7: Relplots of goal trends

For this section of the analysis we separated EPL and Man United data and compared average goals scored in EPL matches against average goals scored by Man United in the EPL. The relplots further corroborate the picture we saw in the Elo analysis. An especially noticeable dip in goals for Manchester United appears at around **2013**, the time when Sir Alex left the club. Though the EPL has seen an increase in average goals over the years, Manchester United cannot seem to go back to their scoring ways of the past. Furthermore, the Goal Trend chart seems to follow a similar pattern to the Elo chart of Man United, indicating that stronger teams tend to score more.

3. <u>Additional Insights from Pandas</u>

We have used pandas functions to aid our exploratory data analysis, however we will try to use some of these functions exclusively in the following section:

3.1. Do home teams have an advantage?

Here, we took **value_counts** of the FTResult feature which has the values H (Home Team win), D (Draw) and A (Away Team win). After normalizing the same we got the following results:

1) **H: 45.49%** 2) **A: 28.88%** 3) **D: 25.63%**

The data indicates clear **home team advantage**, with home teams winning around 17% more games than away teams. This suggests that crowd support and venue familiarity can influence the outcome of a game.

3.2. Has This Home Team Advantage Trend Changed Recently?

Now we attempt to see if this home team advantage is due to old data, or if the trend is prevalent even now. We made use of **query** function to select data for **Years after 2020**. Then, after following the same process for the rest we found that:

1) **H: 43.12%** 2) **A: 31.79%** 3) **D: 25.09%**.

The data suggests there is **no significant difference** in the scenario but shows a more balanced distribution, indicating factors like improved travel conditions and tactical adaptations may be reducing traditional home team advantage in modern football.

3.3. Teams named United in EPL

As the name *United* appears every now and then in the EPL, I wanted to see how many teams there actually have had United in their names in the past 25 years. I used **re.compile** to write the **regex** code to look for team names with United (*\bUnited\b)*. Upon using the **loc** function to search for team names with this regex pattern, I found that only 2 teams come up: Man United and Sheffield United. This seemed wrong and upon checking the dataset I found that some of the teams do not have their full names in the dataset. For example: Newcastle United is called just Newcastle.

3.4. Highest scoring games of Manchester United

	MatchDate	ManUtdGoals	OpponentGoals	Opponent
1	2021-02-02	9	0	Southampton
2	2011-08-28	8	2	Arsenal
3	2010-11-27	7	1	Blackburn
4	2001-12-22	6	1	Southampton
5	2020-12-20	6	2	Leeds
6	2003-04-12	6	2	Newcastle
7	2001-02-25	6	1	Arsenal
8	2000-09-05	6	0	Bradford
9	2008-01-12	6	0	Newcastle
10	2006-08-20	5	1	Fulham

Table 1: Highest Scoring Man United Games

After looking at the bleak reality of Manchester United's current state so far. We will now end this section of our analysis by peaking at the club's best days through finding the top 10 highest scoring games of Manchester United. We created new columns to display opponent names and goals. Then we sorted the df by ManUtd goals and set ascending=False to get the top values.

Here we see that he **largest margin** Manchester United won by surprisingly happened in 2021, a time when Man United was not doing too well. I remember watching the game and it was made possible by the fact that a Southampton player picked up a red card in the first minutes of the game. The **second highest scoring game** is iconic as well. It was when Man United beat one of their biggest rivals, Arsenal, who faced their heaviest league defeat since 1927.

4. Modeling

Given the diversity of our dataset, we have to take one of 2 approaches:

1. Reduce features to preserve maximum number of rows.
2. Preserve features at the cost of working with less rows.

For this project, I decided to try both and compare their performances.

4.1. Modeling Part 1 (Preserving Maximum Number of Rows)

We need to do additional **data cleaning and preparation** to make the data fit our approach of prioritizing rows.

- We will subset our original dataset and take **32** divisions (Excluding only 3rd tier leagues for their lack of relevant data) and make df_modeling data frame with 174823 rows x 38 columns
- We check for null values and drop all features with more than 50% missing values to preserve rows. Then we use dropna() to clean the remaining dataset by removing rows from the remaining columns which had around 10% missing values and were left with **100016 rows × 16 columns**
- For our **target variable,** we **encoded** the categorical values of FTResult feature which now has two unique values, 1 (When home team wins) and 0 (When home team draws or loses)
- We also encoded the HTResult feature into 0 and 1

Upon checking, we saw that our data well balanced at nearly 50/50 proportions of 1 and 0 for the target. This dataset will have **predictive capabilities** as most features are available before the game and upon getting the values for Half-Time score feature our model can predict at half time if the home team will win the game or not. Our final dataset dimensions are **100016 rows × 16 columns**

Train Test Split: To train models and test the same on data we used train_test_split from sklearn. We separated our features and target variable in X and y and then to bring them into the same scale, we used **MinMaxScaler** and **scaled all our features** in the range 0 to 1 (All our categorical features were already encoded during cleaning). Then we split the training and test data into 80:20 proportions while preserving the stratification of the original dataset in out training data for more accurate testing conditions. We also set random state parameter to 42 to ensure consistent results every time we run our models.

Metric | Model	LR Model 1	SVM 1	Decision Tree 1	Random Forest 1
Accuracy	74.97%	**75.52%**	75.03%	74.29%
Precision	**76.23%**	77.31%	75.42%	74.17%
Recall	64.15%	61.05%	**65.69%**	65.42%

Table 2: Summary of Model Performance Metrics 1

Logistic Regression Model 1: Our LR model will do **binary classification** whether the home team will win or not for the given set of features. It predicts the probability that a given input belongs to a particular category by modeling the relationship between a set of independent variables and a dependent variable. The model works by fitting a linear combination of the input features, weighted by coefficients, and passing it through the logistic (sigmoid) function to produce probabilities. Our **model coefficients** show that **half time score, match odds** and **elo** most significantly affect the probability of the home team winning a game. This shows that teams **leading at half time** and with **better odds** are favorable to win. While this model achieved the **highest precision** among the models, it is prone to **Type II error** as evidenced by the low recall, showing it can identify home wins around 64.15% of the time. We also get an **AUC** of 73.9%.

Support Vector Machine 1: The **SVM** model works by finding the optimal hyperplane that best separates data points of different classes, maximizing the margin, which is the distance between the hyperplane and the nearest data points (called support vectors) from each class. For our data the model will go through the 100,016 rows of data and use the independent features to create a hyperplane that separates home team wins (1) and home team draws/losses (0). Our **model accuracy** jumped from **65%** to over **75%** after using **MinMaxScaling** on the features, indicating the model is sensitive to the magnitude of values and scaling it uniformly produces better results. This model gives the best accuracy of all the models in this section but has weaker recall than all the models, indicating that the hyperplane cannot separate values optimally.

Decision Tree 1: A decision tree works by splitting the input data into subsets based on feature values, creating a tree-like structure where each internal node represents a decision based on a feature, each branch represents the outcome of that decision, and each leaf node represents a final prediction or class label. The splits are chosen to maximize **information gain** based on metric like **Gini Impurity**. In our model the tree starts at **HTResult** node and splits to betting odds like **MaxAway** and **MaxHome** which gives the maximum betting odds for away or home win. The model tries to reduce Gini Impurity until it reaches a pure node, however letting the model continue indefinitely might lead to **overfitting**. After trial and error we got the best accuracy for MaxDepth equal to 7. The model performs similarly to other ones but has **better recall**.

Random Forest 1: We run an additional model to tackle the overfitting issue of Decision Trees. RF model is built on the concept of decision trees. It operates by constructing multiple decision trees during training each on a random subset of the data (through processes like bootstrapping) and a random subset of features at each split, to **reduce overfitting** and improve generalization. The final prediction is made by aggregating the outputs of individual trees. For our model we used the same train-test data set n_jobs=-1 (a parameter that tells the algorithm to use all the available processors computer to speed up the training process), but the achieved performance was not better than the Decision Tree itself.

Findings: The model performances indicate **the inherent unpredictability** of the game itself. It is not easy to accurately predict outcomes based on some features. A more specialized model focusing on a specific league may be more accurate, but among our models trained on the **maximum amount of data possible**, the **SVM model is the most accurate**, although the LR and DT models have better recall.

4.2. Modeling Part 2 (Preserving Maximum Number of Features)

We need to **prepare and clean the original dataset** again to fit our approach of preserving rows now:

- We subset the same 32 leagues as before. We remove lower leagues to lack of data
- We drop 10 redundant/unimportant columns and are left with 28 features to work with
- We apply dropna to remove the null values in the remaining columns. We lose rows of data but our approach requires us to preserve columns. We are left with **69137 rows × 28 columns**

We also do the same **Train Test Split** operations as before and use **MinMaxScaler** to scale all our features as all the new rows include numerical values. The models in this section will lack predictive abilities as we use features of in game stats like: Corners, Shots taken and Red cards. Though the models here cannot predict outcome, this can explain **how different statistics affect the outcome** of a game.

Metric | Model	LR Model 2	SVM 2	Decision Tree 2	Random Forest 2
Accuracy	**78.38%**	77.42%	76.33%	77.46%
Precision	**77.6%**	77.53%	76.79%	77.24%
Recall	**72.08%**	69.19%	66.91%	69.80%

Table 3: Summary of Performance Metrics 2

Logistic Regression Model 2: The inclusion of features related to in-game statistics seem to significantly improve the LR model. In addition to the previously significant features, **model coefficients** of HomeRed (-2.163981) and AwayRed (5.47292) seem to significantly reduce and increase odds of home team winning respectively. This makes sense, as the opponent losing a man is advantageous for the home team and vice versa. Interestingly, the away team getting a yellow card seems to reduce odds of home victory a little, this could be due to the fact that opponents often play aggressively to protect an established lead. The model is more accurate than our first LR model and significantly reduces **Type II error** with a 8% better recall and it has **an improved area under the curve** at **77.43%.**

Support Vector Machine 2: The SVM model also shows improvements across all metrics, with the most significant jump coming in recall which improved by 8%. The hyperplane can more accurately separate between 0s and 1s likely due to the inclusion of features that show in-game statistics.

Decision Tree 2: While the decision tree metrics are slightly improved, the model performance is not significantly better compared to the first scenario. Here the nodes split from HTResult to other in game statistics nodes like shots on target. The increased features may be confusing for the model and it has the risk of overfitting, which is why the performance may not have increased much.

Random Forest 2: The new Random Forest Model shows significantly better performance compared to both the **Decision Trees**. This displays the strengths of random forest as it aggregates results from multiple decision trees to reduce bias and overfitting. The previous RF model may not have worked so well due to the diversity of data we had while preserving rows. But here, the model performs better with more features.

Findings: While the models are not perfect, and the **inherent unpredictability** of football also plays a role here. We see that model performances have increased overall with the inclusion of features that have in game statistics. For this dataset, having more data points to work with did not necessarily improve the model performance **as the models in our second part performed better with less rows, but more features.** However, the models in the second part lose the **predictive capability** of the first set of models as we need in-game statistics here. This is more useful to understand the impact of certain stats on games, as we saw how fouls and cards affect the outcome of a game. Lastly, we compare all the models so far.

4.3. Model Summary

Approach | Model	LR	SVM	Decision Tree	Random Forest
Preserving Rows (Accuracy)	**78.38%**	77.42%	76.33%	77.46%
Preserving Features (Accuracy)	74.97%	75.52%	75.03%	74.29%

Table 4: Comparison of Model Accuracy

A comparison of all the models so far reveals that our **second Logistic Regression** model after taking the second route of **preserving features**, is the **most accurate**. It also gives the best performance in terms of **precision** and **recall**. However, what model is *better* depends on the context. In our case, though the Logistic Regression model trained on less rows, but higher features **(69137 rows × 28 columns)** fared better than the LR model trained on more datapoints but less features **(100016 rows × 16 columns)**, we must remember that the improved model **lost its predictive ability** of the first model. If we want to predict match outcomes, the **first SVM Model** is more useful. For someone trying to analyze the impact of different statistics on the match outcome, the **second LR Model** is the best option.

This is also a great demonstration of how these models work. Though I initially split my approach into **prioritizing rows** vs **prioritizing columns** to see which makes the best model. But I found that an additional tradeoff lies in the prediction ability. Though our second set of models had fewer rows of data, we gained accuracy due to the inclusion of in game statistics, which understandably affect match outcomes. Another interesting observation was how the **random forest and decision tree model performances varied** during the 2 different approaches. When we were **working with more rows**, the decision tree worked well when dealing with less features, but the random forest model could not do any better. But when we **worked with more features**, the decision tree model could not improve much but the random forest model showed significantly improved results across all metrics.

5. Reflection

The first and foremost realization I had while working on this project is that **real-world data is messy**. Though it was exciting for me to work with data on a topic I enjoy, finding right ways of dealing with null values and wrong datatypes and confusing features proved difficult. As a result, I had to constantly clean and prepare the data for different sections of the report. Data cleaning alone proved to be the most time-consuming part of the report. Despite these hurdles, the insights discovered were rewarding. To see how different leagues compare and to see the story of my favorite club has been worth the time spent cleaning and extracting the right data. The 2-way approach in the modeling section further helped me understand how data and models truly work and the differences across models. I feel like I have learned a lot over the course of this project through trial and error during coding the whole thing. It was not possible to include all the intricacies in this report, but the broad overview is available in the ipynb file. Overall, the project reinforced my notion that data cleaning and iterative tuning is paramount in producing meaningful results.

References

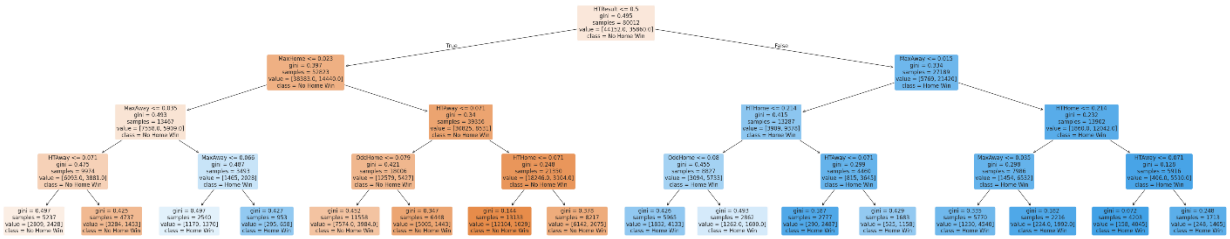
Gábor, A. (2025). Club Football Match Data. Retrieved from https://github.com/xgabora/Club-Football-Match-Data-2000-2025/

NY Times. (2025). European football revenue hit record €38bn *The Athletic*. https://www.nytimes.com/athletic/6419926/2025/06/11/european-football-revenue-premier-league/

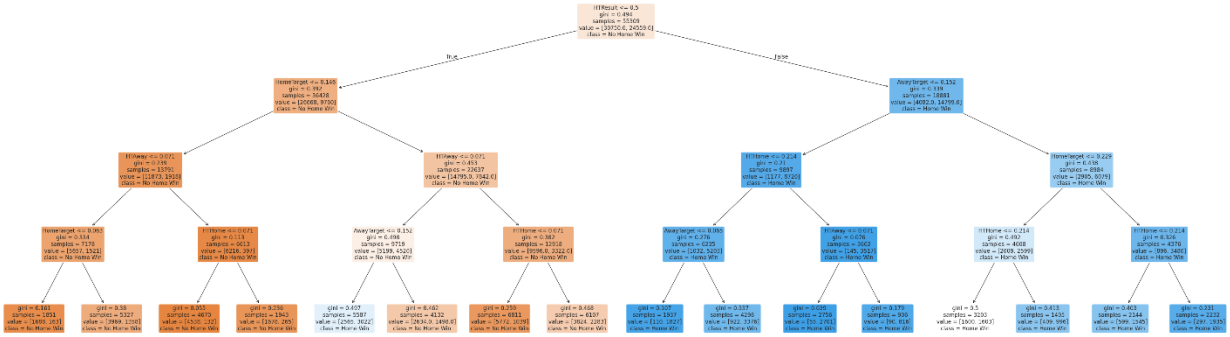
Furniss, M. (2025). *The Strongest Leagues in the World: Insights from the Opta Power Rankings*. https://theanalyst.com/articles/strongest-leagues-in-the-world-opta-power-rankings-june-2025

Appendix

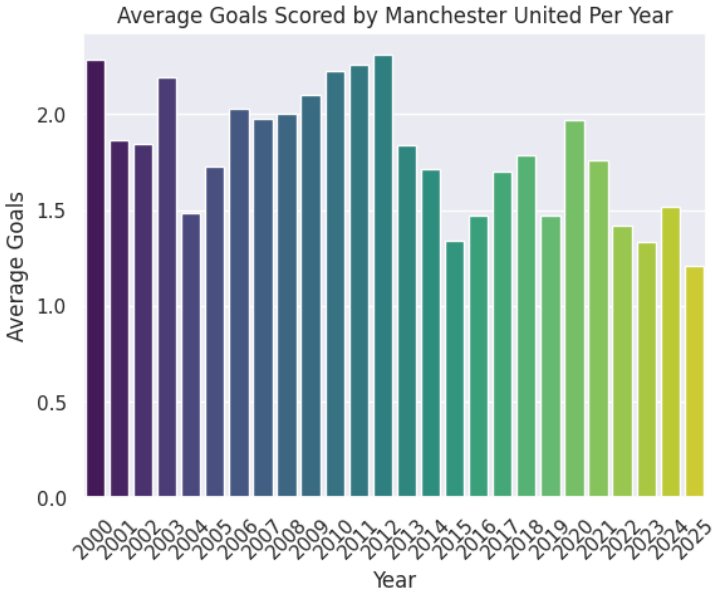
1. Decision Tree 1



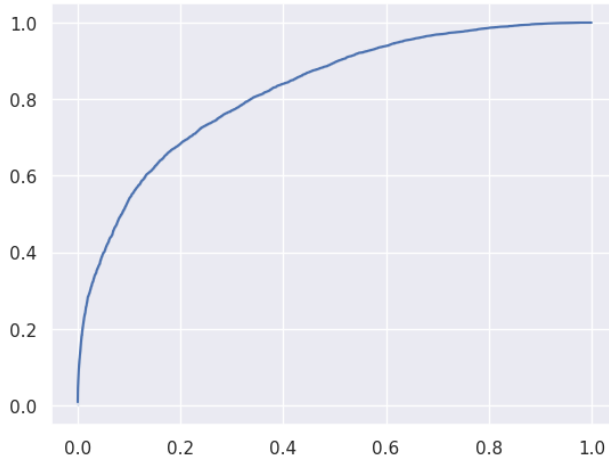
2. Decision Tree 2



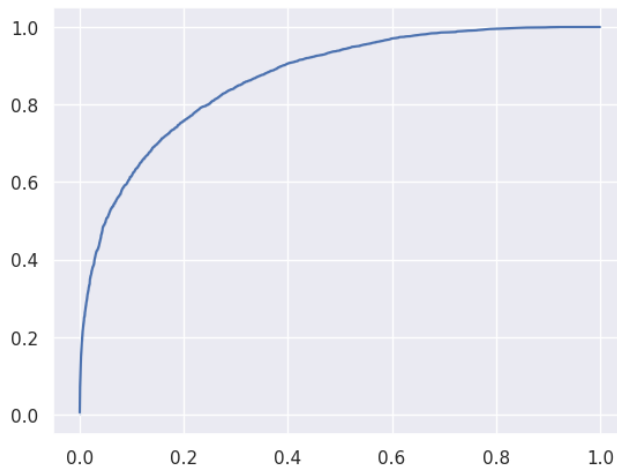
3. Average goals of Manchester United over the years



4. ROC curve (Logistic Regression 1)



5. ROC curve (Logistic Regression 2)



6. Descriptive statistics of total goals in a match across leagues

Division	count	mean	std	min	25%	50%	75%	max
English Premier League	9325.0	2.717748	1.667860	0.0	2.0	3.0	4.0	11.0
French Ligue 1	8630.0	2.493395	1.594809	0.0	1.0	2.0	3.0	10.0
German Bundesliga	7447.0	2.949107	1.717791	0.0	2.0	3.0	4.0	11.0
Italian Serie A	8891.0	2.677877	1.649673	0.0	1.5	3.0	4.0	10.0
Spanish La Liga	8918.0	2.674591	1.682479	0.0	1.0	2.0	4.0	12.0

7. Description of features

Feature	*dtype*	Description
Division	*enum*	League that the match was played in - country code + division number (*I1 for Italian First Division*). For countries where we only have one league, we use 3-letter country code (*ARG for Argentina*).
MatchDate	*date*	Match date in the classic YYYY-MM-DD format.
MatchTime	*time*	Match time in the HH:MM:SS format. CET-1 timezone.
HomeTeam	*string*	Home team's club name in English, abbreviated if needed.
AwayTeam	*string*	Home team's club name in English, abbreviated if needed.
HomeElo	*float*	Home team's most recent Elo rating.
AwayElo	*float*	Away team's most recent Elo rating.
Form3Home	*int*	Number of points gathered by home team in the last 3 matches (*Win = 3 points, Draw = 1 point, Loss = 0 points, so this value is between 0 and 9*).
Form5Home	*int*	Number of points gathered by home team in the last 5 matches (*Win = 3 points, Draw = 1 point, Loss = 0 points, so this value is between 0 and 15*).
Form3Away	*int*	Number of points gathered by away team in the last 3 matches (*Win = 3 points, Draw = 1 point, Loss = 0 points, so this value is between 0 and 9*).
Form5Away	*int*	Number of points gathered by away team in the last 5 matches (*Win = 3 points, Draw = 1 point, Loss = 0 points, so this value is between 0 and 15*).
FTHome	*int*	Full-time goals scored by home team.
FTAway	*int*	Full-time goals scored by away team.
FTResult	*enum*	Full-time result (*H for Home win, D for Draw and A for Away win*).
HTHome	*int*	Half-time goals scored by home team.
HTAway	*int*	Half-time goals scored by away team.
HTResult	*enum*	Half-time result (*H for Home win, D for Draw and A for Away win*).
HomeShots	*int*	Total shots (*goal, saved, blocked, off-target*) by home team.
AwayShots	*int*	Total shots (*goal, saved, blocked, off-target*) by away team.
HomeTarget	*int*	Total shots on target (*goal, saved*) by home team.
AwayTarget	*int*	Total shots on target (*goal, saved*) by away team.
HomeFouls	*int*	Total fouls by home team.
AwayFouls	*int*	Total fouls by away team.
HomeCorners	*int*	Total corners taken by home team.
AwayCorners	*int*	Total corners taken by away team.
HomeYellow	*int*	Total yellow cards awarded to home team players (*excl. staff*).
AwayYellow	*int*	Total yellow cards awarded to away team players (*excl. staff*).
HomeRed	*int*	Total red cards awarded to home team players (*excl. staff*).

AwayRed	*int*	Total red cards awarded to away team players (*excl. staff*).
OddHome	*float*	Bet365's Home Team Win Odd.
OddDraw	*float*	Bet365's Draw Odd.
OddAway	*float*	Bet365's Away Team Win Odd.
MaxHome	*float*	Maximum Home Team Win Odd from ~17 European bookmakers.
MaxDraw	*float*	Maximum Draw Odd from ~17 European bookmakers.
MaxAway	*float*	Maximum Away Team Win Odd from ~17 European bookmakers.
Over25	*float*	Bet365's Over 2.5 Total Goals Scored Odd.
Under25	*float*	Bet365's Under 2.5 Total Goals Scored Odd.
MaxOver25	*float*	Maximum Over 2.5 Total Goals Scored Odd from ~17 European bookmakers.
MaxUnder25	*float*	Maximum Under 2.5 Total Goals Scored Odd from ~17 European bookmakers.
HandiSize	*float*	Asian handicap size for home team (*negative number indicating stronger home team*) .
HandiHome	*float*	Bet365's Home Team Win Odd with the given handicap size for Home team.
HandiAway	*float*	Bet365's Away Team Win Odd with the given handicap size for Home team.
C_LTH	*float*	Likeliness of match falling into the Low Tempo Home-oriented match cluster.
C_LTA	*float*	Likeliness of match falling into the Low Tempo Away-oriented match cluster.
C_VHD	*float*	Likeliness of match falling into the Visibly Home Dominated match cluster.
C_VAD	*float*	Likeliness of match falling into the Visibly Away Dominated match cluster.
C_HTB	*float*	Likeliness of match falling into the High Tempo Balanced match cluster.
C_PHB	*float*	Likeliness of match falling into the Highly Physical Balanced match cluster.